

梦熊炼石·S/N30连测

contest20250904 第三场

命题人：黄佳旭；讲解：葛潇

T1.委托

一个 $n+1$ 个节点的菊花图，中心为0号，点 i 到0的边权为 t_i ， $i=1..n$ 。只允许在特定时间 a_j (出发)从 $0 \rightarrow x_j$ ，或在 b_k 从 $y_k \rightarrow 0$ ($j=1..m, k=1..L$)，不能有任何其他方式移动。问初始在0总共最多能走多少条边

$1 \leq n, m, L \leq 100000$, $0 \leq t_i, a_j, b_k \leq 1e9$

T1.委托

→ 如果去了点 $i > 0$, **必须尽快回来**, 才可能继续走

对任何 $j=1..m$, 求其最早回到0点的时间: $R_j = \min_{b_k > a_j + t_{x_j}} b_k + t_{x_j}$

这容易二分预计算得到

→ 接下来问题就只集中在0号的时间线了

有 m 个区间 $[a_j, R_j]$, 求最多的无交集的区间子集。这是经典贪心场景

→ 唯一要注意的**最后一次可以一去不返** ($R_j = \infty$)

这一次的收益为1, 而其他区间都是 (往返) 收益2, 两者不能统一贪心
只要贪的时候从 (时间轴) 大到小

根据第一个区间权重是1还是2分两类单独做即可

复杂度: $O(n \log n)$ (不细区分 n, m, L)

T2.骑车环行

给定 n 个点 m 条边的简单连通无向图，点权为 h_i ，边权 $w_{uv} = |h_u - h_v|$ 。
求对任意 $i=1..n$ ，**经过 i 的最长边最小的简单环**（孤立点不算）

$1 \leq n \leq 100000$, $1 \leq m \leq 200000$, $0 \leq h_i \leq 1e9$

T2.骑车环行：部分分

→ 首先是不是简单环其实不怎么影响

唯一的限制是**进出起点不能是同一条边**

→ $n \leq 1000$, $m \leq 2000$ (20分)

先Dijkstra预计算两两最短路，再**任选2条相邻的边** $(i, j)(j, k), i \neq k$

令 $ans_j = \min(ans_j, \max(w_{ij}, w_{jk}, D_{ik}))$ 。复杂度： $O(nm \log m + m^2)$

这里发现**枚举起点不是好办法，通过其他方式得到环再更新ans较好**

→ $n=m$ (基环树, +10分)

侧链上显然无解，环上的答案就是环的答案 (额)

加上后面 $m \leq n + 10$ 的暗示：**要考虑生成树与生成树之外的边 (树环)**

T2.骑车环行：部分分

→ $h_i \leq 1$ (+15分)

答案只有0,1,无解, **枚举答案**

只保留0边, 类似拓扑排序那样裁掉树形部分, 剩余点 $ans=0$

保留01边再做一遍, 得 $ans=1$ 的点

发现可以直接推广到 $h_i \leq 10$ (再+15分)

→ 这过程类似Kruskal那样逐边加入并更新答案

区别是Kruskal维护动态连通性

这里要求维护**动态双连通性**

T2.骑车环行：正解

→ 经以上铺垫，如下结论就不算太突兀了

对任何成为最优解的环，有且仅有1条边不在MST上

否则将一条非树边替换成树上路径，并缩掉自交形成的环

由于另一条非树边的存在，不会缩成孤立点

→ 再根据（最长边）MST的特征，如上环的最优解就是那条非树边的长度

对每条非树边，用其边权更新一下对应的那条树上路径即可

暴力实现： $O(m \log m + (m - n)n)$ ，可得 $m - n \leq 10$ 的共30分

→ 经典的树上路径更新求min

无脑树剖+线段树： $O(m \log m + m \log^2 n)$ （不确定能否卡过？）

→ 非树边按从小到大（Kruskal那种顺序）加，则第一次更新到就是最优解

每更新1条路径，就（用并查集）将其缩成一个点

则主体过程可 $O(n + m)$ ，具体实现细节参见标程

T3.数列涂色

给定排列 $a_1 \dots a_n$ ，求不相交的一个上升子序列和一个下降子序列，使总长度最大。 $T \leq 5$ ， $1 \leq n \leq 100000$

T3.数列涂色

→ 如存在不相交的LIS,LDS, 那就是最优解 (废话)

否则两者最多也就相交1次, 即 $ans \leq |LIS| + |LDS| - 1$

→ 问题转为**判断是否任意LIS和LDS都相交**, 这通过对比计数就行了

1. 单独求出LIS、LDS的计数 C_I, C_D

2. 求相交的LIS,LDS有几对, 可通过交点分类,
即(相交于) a_i 的LIS/LDS有几对 (1个X型结构)

→ 预计算:

$L_{i,1..4}$ = 以i结尾/开头的最长上升/下降子序列长度

$c_{i,1..4}$ = 对应↑的最优方案数

→ 则ans要减一的判据为

$$C_I C_D = \sum_{i=1..n} c_{i1} c_{i2} c_{i3} c_{i4} [L_{i1} + L_{i3} = LIS - 1] [L_{i2} + L_{i4} = LDS - 1]$$



T3.数列涂色

→ 如何预计算？以 L_{i1}, C_{i1} 为例（其实是板子）

从1..n遍历i，维护 f_v = 当前以v结尾的最长上升子序列长度

$$L_{i1} = 1 + \max_{v=1..a_i-1} f_v, \text{ 算出后更新 } f_{a_i} = \max(f_{a_i}, L_{i1})$$

用BIT维护 $f_{1..n}$ 即可

→ 考虑最优解计数时，改为 $f_v = (L, C)$ ，L=长度，C=计数

$$\text{并定义 } \max((L_1, C_1), (L_2, C_2)) = \begin{cases} (L_1, C_1) & L_1 > L_2 \\ (L_2, C_2) & L_1 < L_2 \\ (L_1, C_1 + C_2) & L_1 = L_2 \end{cases}$$

BIT的查询和更新都改用以上max求法即可

T4.双端队列

维护n个节点的森林，开始都是孤立点，支持如下操作共m次

- 1.2.将点x添加为点y的第一个/最后一个儿子节点（保证x,y是根节点）
- 3.4.将点x的第一个/最后一个子树切割下来（保证x是根节点）
- 5.询问当前情况下，**如要**通过操作**3,4**使点x成为根，至少要操作几次

$1 \leq n, m \leq 200000$

T4.双端队列：部分分

→ 首先要搞清楚操作5求的是什么

因为只能从根节点切割，为使 x 为根，首先要让 fa_x 为根
即 $ans_x = ans_{fa(x)} + N_x$ ， N_x 表示 x 左右兄弟数中小的那个

→ 暴力实现： $O(nm)$ (45分)

→ 没有3,4类操作（只合不分，+15分）

可以先根据最终状态求一个DFS序

整个过程中各树DFS序都是其一个区间（操作1,2做的就是两段拼接）

→ 在此最终DFS序上维护操作5（当前的）的答案

当 x 被合并到 y 后， **y 的前/后一半的子树上所有答案+1**

（DFS序上区间加，线段树维护）

而树结构就暴力维护就行了（如暴力合并邻接表）

→ 复杂度： $O(n + m \log n)$

T4.双端队列：正解（口糊思路）

- 还是考虑在DFS序上区间加来维护目标函数
此时无法预先给每棵树固定（全局的）DFS访问顺序
但每棵树内部还是连续的一段，且合并/分裂不会改变这一点
- 那么可以给每棵树维护一个平衡树表示相对大小意义下的DFS序
则操作1~4对应两个（有整体先后的）平衡树拼接
或从一个平衡树上分裂出去一个前缀/后缀
- 各平衡树上还可通过类似线段树的tag方式实现
合并：区间+1；分裂：区间-1
- 平衡树要支持合并/分裂+Tag，FHQ是天选实现
- 主体图像是一些(用平衡树维护的)区间在互相合并/切割
为了算切割/段更新的分割点位置
还需存一些原树结构上的信息（如儿子个数）

T4.双端队列：正解（部分实现细节）

→ 标程中维护了两个FHQ

一个叫T，另一个代码里无名（全局的），这里称为X。main→X→T

T.val(别名tsz)：该节点在原的**子树大小**

T.sum：T.val在TreapT上的子树和（区间对应的原树的节点总数）

T.ct(别名cs)：该节点在原树的**儿子数**

T.sz：该Treap子树(区间)包含所在根节点的几个子节点

X.va：最终(操作5)的目标函数

X.tg：X.va区间加的tag

X.fa：该节点在TreapX的父节点

X.sz：TreapX上的子树大小（区间节点数）

→ 以pushback为例，先在两个FHQ上merge

再在T上查出后一半子树的分割点，然后打上tag+1

复杂度： $O((n + m)\log m)$

T4.双端队列：更多细节

→ **T.sz**这个目标函数最关键（也最微妙）

其在T合并时会被强制设成1

使再下层的sz不会透传过来，保证只计数“亲儿子数”

如又分裂出去，会通过pushup重新从Treap下层捡起原来的值

→ 代码中最终问询是从x点本身（在TreapX）自底向上算

所以沿途可能还有未下放的tag，要顺手累加上

（应该也可以自顶向下实现？）

```
void pushback(int x, int y) {  
    sz[y] = 1;   
    sum[y] = val[y];  
    ++ct[x]; // y作为原树上x的最后一个儿子  
    val[x] += val[y]; // 原树节点数合并  
    rt[x] = merge(rt[x], y);  
}
```

```
int popback(int x) { // 分离最后一个子树  
    int y;  
    --ct[x];  
    split(rt[x], ct[x], rt[x], y);  
    val[x] -= val[y]; // 原树上拆开  
    return y;  
}
```

T4.双端队列：更多细节

→ 为何不能合一个平衡树？

查询分割点的时候，FHQ也是通过本身merge/split实现的
这就会导致循环影响（如查size的过程中size变了）